

How the Multi-Scalar PDF Project Works

**A complete, jargon-free walkthrough of the MPI data-sampling pipeline
and the Dirichlet Mixture Density Network**

Generated documentation for the `Multi-ScalarPDF` workspace

August 27, 2026

Who this document is for

You have never worked with turbulent combustion, probability distributions, or neural networks. You want to understand *exactly* what this code does and why, without having to look up a single technical term. Every specialised word in this document is explained the first time it appears, and again in the glossary at the end. Every time we describe something the code actually does, we tell you the file and the line number so you can go look at it yourself.

How to read the source references

Throughout the text you will see markers like `process_work_units_on_rank`[MPI:640]. The monospaced part is the name of a function in the code; the grey bracket is a short file tag plus the line number where that function begins. If you open this PDF on the machine that holds the code, clicking the name jumps straight to that line in VS Code. The file tags are:

Tag	File
MPI	EnightPDFHybridDatasetMPI.py
SER	EnightPDFHybridDataset.py
GRID	dirichlet_mdn/bin_grid.py
DATA	dirichlet_mdn/data.py
SPLIT	dirichlet_mdn/splits.py
MODEL	dirichlet_mdn/model.py
LOSS	dirichlet_mdn/losses.py
TRAIN	dirichlet_mdn/train.py
EVAL	dirichlet_mdn/evaluate.py
METRIC	dirichlet_mdn/metrics.py
PLOT	dirichlet_mdn/plotting.py
VERIFY	dirichlet_mdn/verify.py

Contents

1	The Problem, in Ordinary Language	5
1.1	What is actually being simulated	5
1.2	The triangle	5
1.3	Two levels of detail: fine and coarse	6
1.4	The central question of the whole project	6
1.5	What a “distribution” looks like here	7
2	The Sampling Pipeline: Bird’s-Eye View	8
2.1	The nine stages	9
2.2	The scale of the thing	9
3	Stage 1–2: Reading and Padding the Simulation Data	11
3.1	How the data is laid out on disk	11
3.2	Undoing the parallel decomposition	11
3.3	Wrapping the cube: periodic padding	12
4	Stage 3: The Sliding Window	13
4.1	What the window is and how it moves	13
4.2	What comes out of each window	13
5	Stage 4: The Five Summary Numbers (Moments)	15
5.1	What each number means	15
5.2	The same numbers, computed a second way	16
6	Stage 5: Building the Histogram	17
6.1	Why the bins are not evenly spaced	17
6.2	Exactly how the bin centres are placed	17
6.3	From centres to edges to areas	18
6.4	Counting the dots	18
7	Stage 6: Describing the Shape of a Histogram	19
7.1	The three global descriptors	19
7.2	The seven regional descriptors	20
8	Stage 7: Deciding What to Keep	21
8.1	The problem: two million near-identical examples	21
8.2	Layer 1: chopping moment space into pigeonholes	21
8.3	Layer 2: measuring how different two histograms are	22
8.4	The four-way decision	22
8.5	Bookkeeping when a replacement happens	23

9	Stage 8: Running on Many Processors	24
9.1	Why parallelism is needed and why it is awkward here	24
9.2	Phase 1: independent greedy selection	24
9.3	Phase 2: the single-processor merge	25
9.4	Combining the accounting	27
9.5	The debugging shortcut	27
10	Stage 9: What Gets Written to Disk	28
10.1	The histograms: HDF5 shards	28
10.2	The facts about each example: Parquet	28
10.3	The settings: JSON manifest	29
11	The Idea Behind the Neural Network	30
11.1	Restating the goal	30
11.2	Why not just predict the 2200 numbers directly?	30
11.3	The alternative: predict a formula, not a picture	30
11.4	What a Dirichlet distribution is	31
11.5	Why a mixture of them	31
12	The Network Itself	32
12.1	Architecture	32
12.2	The two output heads	32
12.3	Size	33
13	How the Network Learns	34
13.1	Part 1: matching the histogram	34
13.2	Part 2: matching the moments exactly	35
13.3	Part 3: keeping the components alive	36
13.4	Numerical hygiene throughout	36
14	Feeding the Network: Dataset and Splits	37
14.1	The dataset adapter	37
14.2	Making an honest train/test split	38
15	The Training Loop	40
15.1	Setup	40
15.2	The optimiser and the schedule	40
15.3	One epoch	41
15.4	Choosing when to stop	41
15.5	What a run directory contains	41
15.6	Devices	42
16	Judging the Result	43
16.1	Turning parameters back into a picture	43
16.2	The four numbers that matter	43
16.3	Slicing the results	44
16.4	The plots	44
17	Built-In Self-Checks	45

18 Running Everything	46
18.1 Setting up the environment	46
18.2 The task list	46
18.3 A recommended first session	47
18.4 Practical notes on the MPI sampler	47
Glossary	48
Appendix: Index of Source Locations	50

Chapter 1

The Problem, in Ordinary Language

1.1 What is actually being simulated

Imagine you pour a drop of red dye and a drop of blue dye into a bucket of water and then stir it violently. At first the two dyes are separate blobs. As you stir, they stretch into ribbons, then into threads, then into a fine mist of colour, and eventually they are mixed so thoroughly that every teaspoon of water looks the same shade of purple.

That process — stirring two substances together in a turbulent fluid — is what this project is about. In the real application the two substances are not dyes but two **chemical scalars**: two quantities that get carried around by the flow, such as the concentration of fuel and the concentration of an additive. We call them **scalar A** and **scalar B**. At every point in space, at every moment in time, scalar A has some value between 0 and 1, and scalar B has some value between 0 and 1.

There is one extra rule that matters enormously for everything that follows. The two scalars, plus whatever is left over (the “background”), must add up to the whole:

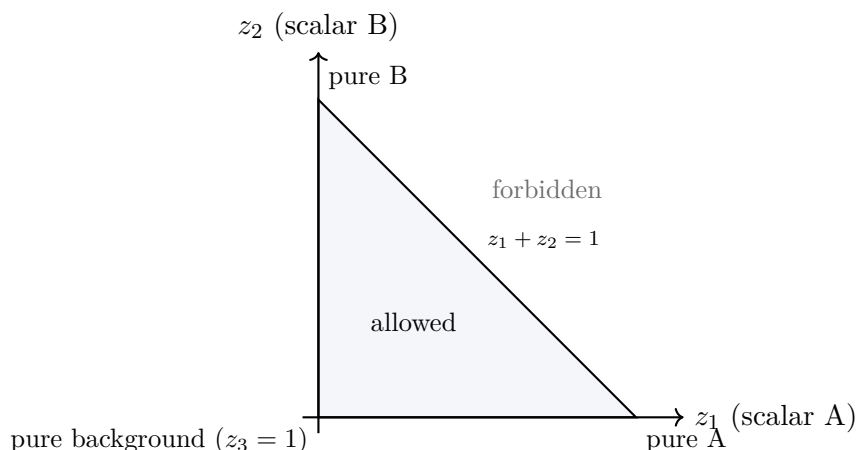
$$z_1 + z_2 + z_3 = 1, \quad z_1, z_2, z_3 \geq 0.$$

Here z_1 is the amount of scalar A, z_2 is the amount of scalar B, and $z_3 = 1 - z_1 - z_2$ is everything else. Because the three numbers are non-negative and always sum to one, only *two* of them are free — once you know z_1 and z_2 , you automatically know z_3 .

In everyday terms. Think of a pie chart with exactly three slices. You can make the slices any size you like, but they must together fill the whole pie. Once you have chosen the size of the first two slices, the third is decided for you.

1.2 The triangle

If you draw all the possible pairs (z_1, z_2) that obey the rule above, you do not get a square — you get a *triangle*. The corners of the triangle are the three “pure” states: all scalar A, all scalar B, all background. The edges are mixtures of exactly two ingredients. The middle is a blend of all three.



Mathematicians call this triangle the **2-simplex**. You do not need that word, but you will see it in the code (`simplex_mask`, `z_simplex`, `num_simplex_cells`), so it is worth knowing that “simplex” simply means “the triangle of allowed values”. The upper-right half of the square, where $z_1 + z_2 > 1$, is physically impossible: it would mean the two ingredients together add up to more than the whole. The code carries a boolean grid called `simplex_mask` that is `True` exactly on the allowed triangle and `False` outside it; it is built in `bin_grid` [GRID:75] and in `get_histogram_layout` [MPI:177], both with the same one-line test [MPI:200]: `(grid_a + grid_b) <= 1.0 + 1.0e-12`. (The tiny `1e-12` is slack so that points sitting exactly on the edge are not thrown away because of rounding.)

1.3 Two levels of detail: fine and coarse

The reference data for this project comes from a **Direct Numerical Simulation**, or **DNS**. That is a brute-force computer simulation of the fluid that resolves every swirl, down to the smallest one that physically exists. In this project each DNS snapshot is a cube of $256 \times 256 \times 256$ grid cells — about 16.8 million little boxes, each holding one value of scalar A and one value of scalar B.

DNS is extraordinarily expensive. Engineers who design real burners cannot afford it. They run a cheaper simulation called **LES** (Large Eddy Simulation), which uses far coarser cells — say $64 \times 64 \times 64$ DNS cells lumped into a single LES cell. An LES cell knows only *averages*: it knows the average amount of scalar A inside it, and how much that amount wobbles, but it has thrown away the fine-grained detail of what is going on inside.

That lost detail matters, because chemistry is nonlinear: the average of a reaction rate is not the reaction rate of the average. To recover the chemistry you need to know the full **distribution** of values inside the cell, not just the average.

In everyday terms. Knowing that the average household income in a city is \$60,000 tells you very little. A city where everyone earns exactly \$60,000 behaves completely differently from a city where half the people earn \$10,000 and half earn \$110,000. The *spread and shape* of the values is the missing information.

1.4 The central question of the whole project

The one sentence that explains everything

Given only a handful of summary numbers about an LES cell (the averages and the wobble), can we reconstruct the full distribution of scalar values inside that cell?

The project answers that question in two halves:

1. **The sampling pipeline** (Chapters 2–10) chops DNS cubes into millions of virtual LES cells, computes both the summary numbers *and* the true distribution for each, throws away redundant examples, and stores the survivors on disk. This is the “textbook” the neural network will study.
2. **The Dirichlet Mixture Density Network** (Chapters 11–16) is a small neural network that reads the summary numbers and outputs a compact mathematical description of the full distribution.

1.5 What a “distribution” looks like here

Take one LES-sized cube of DNS data, say $64^3 = 262,144$ fine cells. Each of those fine cells gives you one pair (z_1, z_2) — one dot inside the triangle. Plot all 262,144 dots and you get a cloud. Some regions of the triangle are crowded; some are empty.

To store that cloud compactly we lay a grid of small squares over the triangle and simply count how many dots land in each square, then divide by the total. The result is a table of $64 \times 64 = 4096$ numbers that add up to exactly 1. This table is called a **histogram**, or in this project a **joint PDF** (“PDF” = probability density function; “joint” = of two variables at once).

Vocabulary you now own

- **Scalar**: a quantity carried by the flow, valued between 0 and 1.
- **Simplex / triangle**: the set of allowed (z_1, z_2) pairs.
- **DNS**: the fine, expensive, reference simulation.
- **LES cell / filter box**: a cube of many DNS cells, treated as one coarse cell.
- **Histogram / joint PDF**: a 64×64 table of fractions summing to 1, describing how the values inside one filter box are spread over the triangle.
- **Moments**: the summary numbers (averages and wobble) — defined properly in Section 5.

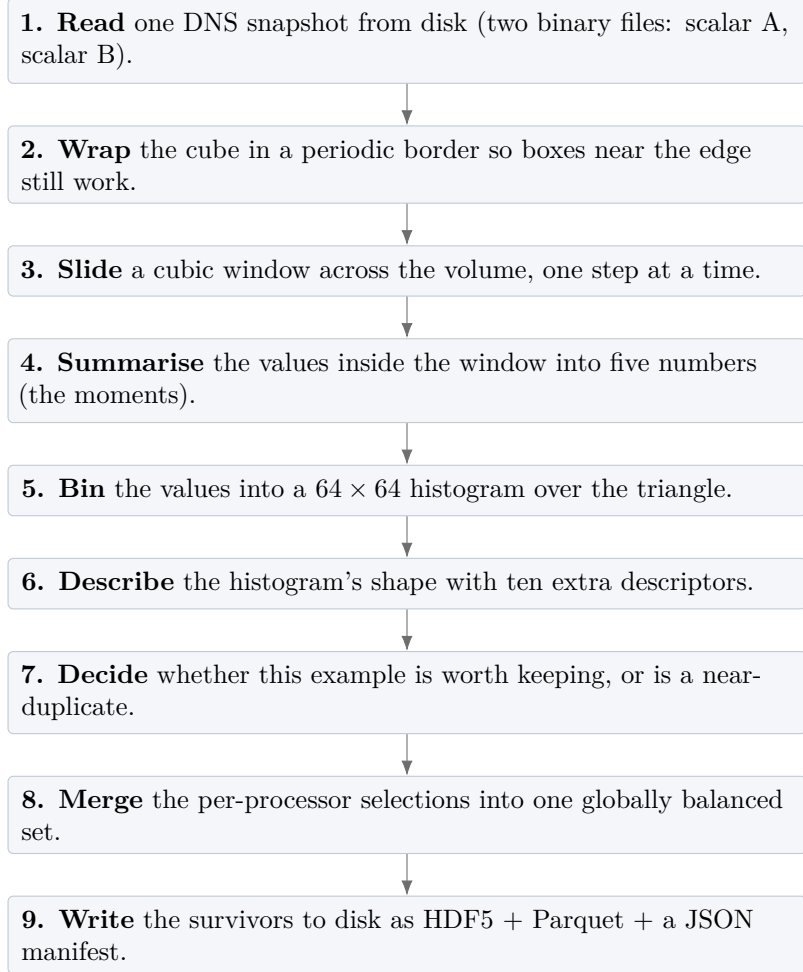
Chapter 2

The Sampling Pipeline: Bird’s-Eye View

The sampling program is `EnightPDFHybridDatasetMPI.py`. Its job is to turn raw simulation output into a curated training set. The word **MPI** in the filename means the program can run on many CPU cores — even many separate computers — at the same time. We will unpack that in Chapter 9.

The whole file is self-contained by design; its opening comment `[MPI:1]` explicitly notes that it deliberately does *not* import the single-process version, so the two scripts can evolve independently.

2.1 The nine stages



Stages 1–7 happen inside `process_work_units_on_rank` [MPI:640]; stage 8 is `merge_per_rank_outputs` [MPI:881]; stage 9 is `materialize_dataset` [MPI:533]. The conductor that calls all of them in order is `main` [MPI:1047].

2.2 The scale of the thing

To make the numbers concrete, here is what the real production run actually did, taken straight from the manifest file it wrote (`hybrid_dataset_mpi/all_cases_w64_w128_manifest.json`):

Quantity	Value
Distinct simulation runs	27
Distinct scalar configurations	8
Timesteps per run	13
Filter box sizes	64^3 and 128^3
Histogram resolution	64×64
Windows examined in Phase 1	2,156,544
Candidates surviving Phase 1 into the merge	825,249
Examples finally retained	165,895
Output HDF5 shard files	7
Wall-clock time of the merge	about 3 hours

So roughly *one in thirteen* of the windows examined survived to the final dataset. Chapter [8](#) explains precisely how that filtering decision is made, because it is the most interesting and least obvious part of the pipeline.

Chapter 3

Stage 1–2: Reading and Padding the Simulation Data

3.1 How the data is laid out on disk

The DNS output lives in a directory tree that looks like this:

```
<root>/run_0000/ensight-3D/ZAI1_0/ZAI1_0.000017
<root>/run_0000/ensight-3D/ZBI1_0/ZBI1_0.000017
<root>/run_0001/ensight-3D/...
```

Reading left to right: `run_0000` is one simulation; `ZA` and `ZB` mark scalar A and scalar B; `I1` is the **scalar configuration** (a label for which initial arrangement of the two dyes was used — the eight labels are listed at [MPI:41]: `I1`, `I4`, `I5`, `L1`, `PI1`, `PI4`, `PI5`, `PL1`); and `000017` is the timestep number. The path construction happens inside `process_work_units_on_rank` [MPI:640], which first checks that the run folder exists, then the `ensight-3D` folder, then the two scalar folders, then the individual timestep files — recording a friendly message in a `missing_paths` list for each thing it cannot find, and only crashing if you passed `--strict-missing`.

Watch out. Missing files are *not* fatal by default. This is deliberate: a 27-run sweep with a few incomplete runs should still produce a usable dataset. But it means you should always read the `missing_paths` section of the output manifest before trusting a dataset.

3.2 Undoing the parallel decomposition

`read_ensight` [MPI:104] does the actual file reading, and it has to undo something the original simulation did.

The DNS was itself computed in parallel, on $8 \times 8 \times 8 = 512$ processors. Each processor owned a small sub-cube of 32^3 cells and wrote its own piece. The file on disk is therefore not a simple “row by row” dump of the big cube; it is 512 small cubes stacked one after another.

The reading procedure is:

1. Skip four header blocks — three 80-byte text banners and one 4-byte integer ([MPI:107]). These hold descriptive text the code does not need.
2. Read the rest of the file as one long list of 4-byte floating-point numbers.
3. Reshape that list into a six-dimensional array with shape $(32, 32, 32, 8, 8, 8)$: the first three indices say “where inside a sub-cube”, the last three say “which sub-cube”. The `order="F"` flag tells the reshape to fill the first index fastest, matching how Fortran (which wrote the file) stores arrays.

4. Loop over the 512 sub-cubes and copy each into its correct slot in a fresh 256^3 array. Note the index order `[:, :, :, kp, jp, ip]` — the processor indices are read back-to-front, which is exactly the transposition needed to undo the Fortran ordering.

The same procedure runs twice, once for scalar A and once for scalar B, and the function returns the two padded cubes.

Watch out. The constants 256 (`--nx`) and 8 (`--npz`) are command-line options, defaulting to the values above. If you point this script at a simulation with a different resolution or a different processor layout and forget to change them, the reshape will either crash or, worse, silently scramble the data. There is no header check that catches this.

3.3 Wrapping the cube: periodic padding

Now a practical problem. We want to slide a 64^3 window over the cube. What happens when the window's centre is near a face of the cube? Half the window would hang off the edge.

The physics gives us the answer for free. These simulations are **periodic**: the cube is a tile in an infinite, repeating pattern. Whatever leaves through the right face re-enters through the left. So we can simply glue copies of the opposite faces onto each side.

`fill_periodic` [MPI:87] does this. It creates a bigger array of size $(256 + 2w)^3$ where w is the padding width, drops the original cube into the middle, and then fills the borders:

- Lines [MPI:92]–[MPI:93] fill the two ends of the first axis by copying the last w slices to the front and the first w slices to the back.
- Lines [MPI:95]–[MPI:96] then do the second axis — but note they operate on `mod`, the already-partially-filled array, not on `orig`. That is what makes the corners come out right without extra code.
- Lines [MPI:98]–[MPI:99] do the same for the third axis.

The padding width is chosen once in `main` [MPI:1047] at [MPI:1088] as `pad_width = max(width // 2 for width in box_widths)` — half of the largest filter box. With boxes of 64 and 128 that gives $w = 64$, so the padded array is 384^3 . In memory that is about 226 MB per scalar as 32-bit floats, or roughly 450 MB for the pair. This is the single largest memory consumer per processor during Phase 1.

In everyday terms. Periodic padding is like the wrap-around in old arcade games: fly off the right edge of the screen and you reappear on the left. Padding just pre-draws that wrap-around so the sliding window never has to think about edges.

Chapter 4

Stage 3: The Sliding Window

4.1 What the window is and how it moves

`iter_filter_records` [MPI:414] is the generator that walks the volume. In Python, a *generator* is a function that hands back results one at a time instead of building a giant list — essential here, because building a list of two million histograms would need terabytes of memory.

The geometry, from [MPI:415] onward:

- `half_width = box_width // 2`. The command-line option `--box-widths` gives the *full* cube edge (e.g. 64), and the code halves it internally.
- Three identical loops run over centre positions `range(pad_width, nx + pad_width, stride)`: they start at the beginning of the real data inside the padded array, run to its end, and step by `stride`.
- For each centre (i, j, k) the window is the slice $[i - h : i + h]$ in each direction — exactly `box_width` cells on a side.

With `nx = 256` and `stride = 32`, each axis yields $256/32 = 8$ positions, so $8^3 = 512$ windows per snapshot for the 64^3 boxes. With `stride = 64` for the 128^3 boxes you get $4^3 = 64$ windows. Multiply by 27 runs $\times$ 8 configurations $\times$ 13 timesteps and you land near the 2.16 million figure in the manifest.

Stride versus box width

`stride` is how far the window jumps between samples; `box_width` is how big the window is. Because the stride (32) is smaller than the box (64), consecutive windows *overlap* by half. Overlapping is intentional — it produces more training examples and smoother coverage — but it also means neighbouring examples are correlated, which is one reason the redundancy filter in Chapter 8 exists.

4.2 What comes out of each window

For every window the generator flattens the two sub-cubes into two long lists of numbers (`z1_values` and `z2_values`, converted to 64-bit precision so the sums stay accurate), then produces:

1. five **moments** computed directly from the raw values,
2. a 64×64 histogram,
3. the same five moments recomputed *from* the histogram, as a cross-check,
4. ten shape descriptors,

5. bookkeeping: which run, which configuration, which timestep, which box width, which stride, the window centre, and the number of DNS cells inside.

All of that is packed into a Python dictionary [MPI:434] and yielded together with the histogram as a pair. One important detail: the centre coordinates recorded in the metadata have `pad_width` subtracted off, so they refer to the *original* cube, not the padded one — see the three `center_i/j/k` entries.

Chapter 5

Stage 4: The Five Summary Numbers (Moments)

5.1 What each number means

`compute_direct_moments` [MPI:215] produces five numbers from the raw values inside one window. In plain language:

Name	Symbol	What it tells you
mean_a	$\bar{z}_1$	The average amount of scalar A in the box.
var_a	$\widetilde{z_1'^2}$	How much the amount of A wobbles from cell to cell. Zero means perfectly uniform; large means very patchy.
mean_b	$\bar{z}_2$	The average amount of scalar B.
var_b	$\widetilde{z_2'^2}$	The wobble of scalar B.
cov_ab	$\widetilde{z_1'z_2'}$	Whether A and B wobble <i>together</i> . Positive: where there is a lot of A there tends to be a lot of B. Negative: they avoid each other. Zero: no relationship.

The formulas, exactly as coded:

$$\bar{z}_1 = \frac{1}{N} \sum_{n=1}^N z_1^{(n)}, \quad \widetilde{z_1'^2} = \frac{1}{N} \sum_{n=1}^N \left(z_1^{(n)} - \bar{z}_1 \right)^2, \quad (5.1)$$

$$\bar{z}_2 = \frac{1}{N} \sum_{n=1}^N z_2^{(n)}, \quad \widetilde{z_2'^2} = \frac{1}{N} \sum_{n=1}^N \left(z_2^{(n)} - \bar{z}_2 \right)^2, \quad (5.2)$$

$$\widetilde{z_1'z_2'} = \frac{1}{N} \sum_{n=1}^N \left(z_1^{(n)} - \bar{z}_1 \right) \left(z_2^{(n)} - \bar{z}_2 \right). \quad (5.3)$$

Here N is the number of DNS cells in the window (64^3 or 128^3).

In everyday terms. The mean is the centre of gravity of the dot cloud in the triangle. The two variances say how far the cloud spreads left–right and up–down. The covariance says whether the cloud is tilted like a diagonal cigar, and which way it leans.

Watch out. The divisor is N , not $N - 1$. Statisticians call this the *biased* or *population* variance. It is the right choice here because the window *is* the whole population we care about, not a sample drawn from a larger one. But if you ever compare these numbers to values produced by another tool, check which convention that tool uses.

5.2 The same numbers, computed a second way

`compute_histogram_moments` [MPI:237] recomputes all five moments, but from the histogram instead of from the raw values. Because the histogram has already thrown away the exact positions of the dots inside each little square, the two answers will differ slightly. The size of that difference is a direct measurement of how much resolution the histogram lost.

The code stores both sets — `mean_a` and `hist_mean_a`, and so on — plus a single worst-case number, `moment_abs_err_max`, which is the largest absolute difference across all five [MPI:451]. If you ever suspect the binning is too coarse, sort the metadata by that column and look at the worst offenders.

Watch out. The neural network is trained on the *direct* moments (the raw-value ones), not the histogram ones. This is a deliberate choice: at deployment time an LES solver would supply true moments, not histogram-derived ones. The histogram moments exist purely as a diagnostic.

Chapter 6

Stage 5: Building the Histogram

6.1 Why the bins are not evenly spaced

The obvious way to divide the interval from 0 to 1 into 64 bins is to make them all the same width, 1/64 each. The code does *not* do that by default, and the reason is physical.

In combustion there is a special mixture ratio, the **stoichiometric** value, at which fuel and oxidiser are present in exactly the proportions the chemical reaction wants. The flame lives there. In these simulations that special value is small — the default is $z_{\text{st}} = 0.1$. Almost all the interesting physics is squeezed into the bottom tenth of the range, and evenly spaced bins would waste most of their resolution on the boring upper region.

So the code uses *fine* bins below z_{st} and *progressively coarser* bins above it.

6.2 Exactly how the bin centres are placed

`get_bin_centers` [MPI:149] builds the 64 bin centres in two pieces. Let $N = 64$ be the number of bins and let $c = N/2 = 32$ be the cut index.

Lower half: evenly spaced. The first 32 centres march linearly from 0 up to z_{st} :

$$\Delta z = \frac{z_{\text{st}}}{c-1}, \quad z_i = i \Delta z \quad \text{for } i = 0, 1, \dots, c-1.$$

With $z_{\text{st}} = 0.1$ and $c = 32$ that is $\Delta z \approx 0.00323$ — bins about twenty times finer than uniform binning would give.

Upper half: a quadratic that speeds up. For $i \geq c$ the centres follow a parabola,

$$z_i = a i^2 + b i + \gamma,$$

whose three coefficients are pinned down by three requirements:

1. the last bin centre must land exactly on 1: $z_{N-1} = 1$;
2. the curve must pass through z_{st} at the joint: $z_{c-1} = z_{\text{st}}$;
3. the spacing must be continuous at the joint, i.e. the derivative there equals Δz , so there is no sudden jump in bin width.

Conditions 1 and 3 give a 2×2 linear system, solved explicitly in the code by Cramer's rule (the variables `m11`, `m12`, `m21`, `m22`, `r1`, `r2`, `delta`); condition 2 then fixes the constant term `coef_c`. The result is a bin layout that starts fine, widens smoothly, and reaches 1 exactly at the last bin.

If you pass `--uniform-bins`, the whole construction is skipped and `np.linspace(0, 1, num_bins)` is used instead.

6.3 From centres to edges to areas

`get_histogram_layout` [MPI:177] takes the centres and produces everything else:

- **Edges.** Interior edges sit halfway between neighbouring centres, $e_i = \frac{1}{2}(z_i + z_{i-1})$. The two outermost edges are extrapolated by reflecting the first and last half-gaps. There are $N + 1 = 65$ edges for $N = 64$ bins.
- **Cell areas.** Because the bins are unequal, each little rectangle in the 64×64 grid has its own area, $A_{ij} = (e_{i+1}^a - e_i^a)(e_{j+1}^b - e_j^b)$. These areas matter enormously later, when we convert between a *probability* (a fraction of dots in a square) and a *density* (probability per unit area).
- **Grids.** `grid_a` and `grid_b` are 64×64 arrays holding the z_1 and z_2 coordinate of every cell centre, produced by `np.meshgrid` with `indexing="ij"` — meaning row index $\rightarrow z_1$, column index $\rightarrow z_2$. Keeping this convention straight is essential; the training code re-derives it in `bin_grid` [GRID:75] and checks the two agree.
- **Simplex mask.** The boolean triangle described in Chapter 1.

Of the 4096 cells in the grid, only about 2200 fall inside the triangle. Everything downstream — the neural network’s loss, the error metrics, the plots — works only on those in-triangle cells.

6.4 Counting the dots

`compute_histogram_pdf` [MPI:224] is deceptively short:

1. `np.histogram2d` counts how many (z_1, z_2) pairs fall into each rectangle, using the edge arrays computed above. `density=False` means give raw counts.
2. If the total count is zero the function raises an error — a guard against empty or corrupted windows.
3. Divide by the total [MPI:234]. The result is a table of fractions that sums to exactly 1.

Watch out. The stored quantity is a *probability mass* per cell, not a probability *density*. The distinction is: $\text{mass} = \text{density} \times \text{area}$. Because the cell areas here vary by a factor of tens across the grid, confusing the two produces badly wrong pictures. The training code is careful about this: see the `log_cell_area` buffer in `HistogramNLL` [LOSS:80], discussed in Section 13.1.

The histogram is converted to 32-bit floats before being yielded, halving its memory footprint. At 64×64 that is 16 kB per example; 825,000 candidates would be about 13 GB, which is why Phase 1 stores them to disk rather than holding them all in RAM.

Chapter 7

Stage 6: Describing the Shape of a Histogram

Two histograms can have identical means and variances and still look completely different — one a single blob in the middle, the other two blobs at opposite corners. To detect this the code computes ten **shape features**, listed by name at [MPI:43].

7.1 The three global descriptors

`compute_shape_features` [MPI:264] first flattens the histogram and computes:

Normalised entropy (`shape_entropy`).

$$H = \frac{-\sum_m p_m \log p_m}{\log M}, \quad M = 4096.$$

Entropy measures spread-out-ness. It is 0 when all the probability sits in one cell and 1 when it is spread perfectly evenly over all cells. Dividing by $\log M$ makes the answer sit between 0 and 1 regardless of grid size. The code clips values at 10^{-16} before taking the logarithm [MPI:266] so that empty cells do not produce $\log 0 = -\infty$.

Peak height (`shape_max_prob`). Simply the largest single value in the table. A tall peak means the mixture is nearly uniform inside the box; a low peak means the values are scattered.

Effective support (`shape_effective_support`).

$$S = \frac{1}{M} \cdot \frac{1}{\sum_m p_m^2}.$$

The quantity $1/\sum p_m^2$ is the “effective number of occupied cells”. If probability is spread evenly over exactly n cells, it evaluates to exactly n . Dividing by M turns it into a fraction of the grid.

In everyday terms. Entropy and effective support both answer “how spread out is this?” but they weight the tails differently. Entropy notices faint, wide tails; effective support mostly ignores them and reports where the bulk of the mass is.

7.2 The seven regional descriptors

`build_shape_masks` [MPI:246] pre-computes seven boolean stencils over the triangle — once, at program start, not per window — and `compute_shape_features` just sums the probability underneath each:

Feature	Region it measures
<code>shape_corner_z1</code>	Near the “pure A” corner: $z_1 \geq 1 - \tau_c$ and $z_2 \leq \tau_c$.
<code>shape_corner_z2</code>	Near the “pure B” corner.
<code>shape_corner_z3</code>	Near the “pure background” corner: both z_1 and z_2 small.
<code>shape_edge_z1_zero</code>	Along the edge where A is absent: $z_1 \leq \tau_e$.
<code>shape_edge_z2_zero</code>	Along the edge where B is absent.
<code>shape_edge_z3_zero</code>	Along the hypotenuse where the background is absent, i.e. $1 - z_1 - z_2 \leq \tau_e$.
<code>shape_center_mass</code>	Within radius r of the triangle’s balance point $(\frac{1}{3}, \frac{1}{3})$.

The three thresholds are command-line options: τ_c is `--corner-threshold` (default 0.08), τ_e is `--edge-threshold` (default 0.04), and r is `--center-radius` (default 0.12). Every mask is intersected with the triangle mask, so no region ever extends into forbidden territory.

Physically, corner mass means unmixed pockets survive; centre mass means the box is well-blended; edge mass means one ingredient is locally missing.

Watch out. These ten features are recorded in the output metadata and are useful for slicing and plotting the dataset. They are *not* used to make the keep-or-discard decision — that uses a different measure entirely, described next.

Chapter 8

Stage 7: Deciding What to Keep

This is the heart of the sampler, and the part most worth understanding.

8.1 The problem: two million near-identical examples

The sliding window overlaps itself, adjacent timesteps look alike, and most of the volume at most times is simply “well-mixed, nothing interesting”. A naive dataset would be 90% redundant, which wastes disk, wastes training time, and — most damagingly — teaches the network that boring cases are what matter.

The solution has two layers: first spread examples evenly across the space of possible moments, then within each region keep only genuinely different-looking histograms.

8.2 Layer 1: chopping moment space into pigeonholes

`compute_moment_bin` [MPI:349] converts the five moments into five small integers, which together form a **bin key**. The recipe per moment:

1. Look up the allowed range [MPI:350]: means run $[0, 1]$; variances run $[0, V]$ where V is `--variance-limit` (default 0.25); covariance runs $[-C, +C]$ where C is `--covariance-limit` (default 0.25).
2. Clamp the value into that range. The clamp uses `np.nextafter(high, low)` rather than `high` itself, so a value sitting exactly at the top does not round up into a non-existent bin.
3. Rescale to $[0, 1)$ and multiply by the number of bins for that moment (`--moment-bins`, default 12,12,12,12,12).
4. Truncate to an integer and clamp again for safety.

With twelve divisions per moment there are $12^5 = 248,832$ conceivable pigeonholes, though only a small fraction ever get used, because real moments are strongly correlated — a box with mean A near 1 cannot also have a large variance.

Each pigeonhole is allowed to hold at most `--max-per-moment-bin` examples (default 50). That single cap is what makes the final dataset balanced: a rare, extreme moment combination gets the same 50 slots as the most common one.

In everyday terms. Imagine sorting photographs into a chest of drawers labelled by subject, and refusing to put more than 50 photos in any one drawer. Even if you took 10,000 pictures of your cat, only 50 make the album — so the album ends up showing your whole life, not just your cat.

8.3 Layer 2: measuring how different two histograms are

Within a drawer we still need to choose *which* 50 to keep. The criterion is visual distinctness, measured by the **Jensen–Shannon distance**, implemented in `jensen_shannon_distance` [MPI:286].

The idea in three steps. Given two histograms p and q :

1. Form their average, $m = \frac{1}{2}(p + q)$.
2. Ask how surprised you would be by p if you expected m , and vice versa. The standard measure of that surprise is the Kullback–Leibler divergence, $\text{KL}(p\|m) = \sum_i p_i \log(p_i/m_i)$.
3. Average the two surprises and take the square root:

$$D_{\text{JS}}(p, q) = \sqrt{\frac{1}{2}\text{KL}(p\|m) + \frac{1}{2}\text{KL}(q\|m)}.$$

The useful properties are: it is 0 exactly when the two histograms are identical; it is symmetric; it is bounded above (by $\sqrt{\log 2}$ in natural units); and it never blows up even when one histogram has empty cells where the other does not — which is why the square root of the *Jensen–Shannon* divergence, rather than plain KL, is the right tool here. The code guards the logarithms by renormalising and then clipping at 10^{-16} [MPI:289].

In everyday terms. Think of it as a “how different do these two pictures look?” score between 0 (identical) and about 0.83 (as different as possible). The default threshold of 0.03 means “if a new picture is at least 3% different from everything already in the drawer, it earns a slot”.

Two speed-critical variants

Comparing a candidate against 50 stored histograms one at a time is slow in Python. Two vectorised helpers exist specifically to avoid that:

- `js_distance_to_stack` [MPI:297] compares one candidate against an entire stack of N histograms in a single batch of array operations, returning N distances at once.
- `pairwise_min_distances` [MPI:323] takes a stack of N histograms and returns, for each one, the distance to its nearest neighbour *within* the stack. This is the “how novel is each thing I already have?” array. It sets the self-distance to infinity [MPI:344] so an item never counts as its own neighbour.

The slower, more readable, one-at-a-time versions — `min_distance_to_existing` [MPI:369] and `get_existing_novelties` [MPI:376] — are used in Phase 1, where bins are smaller.

8.4 The four-way decision

`decide_retention` [MPI:388] contains the actual rule. Given a candidate histogram, its metadata, the list of examples already in its drawer, and a tally of which (`configuration`, `run`, `timestep`, `box width`) sources are already represented there, it returns four things: keep or not, an index to overwrite (or nothing), the measured novelty, and a text reason. The reason string is stored in the output metadata as `retain_reason`, so you can audit every decision after the fact.

The rule reads:

Situation	Decision	Reason recorded
Drawer is empty	Keep it, no questions asked	<code>first_in_bin</code>
Drawer has room, and the candidate is at least <code>shape_threshold</code> away from everything in it	Keep it	<code>novel_shape</code>
Drawer has room, candidate looks similar, but it comes from a source not yet represented in this drawer	Keep it	<code>source_diversity</code>
Drawer has room, candidate looks similar, source already represented	Discard	<code>redundant_shape</code>
Drawer is full, and the candidate is more novel than the drawer's <i>least</i> novel occupant — and is itself either genuinely novel or from a new source	Replace that occupant	<code>replaced_low_novelty</code>
Drawer is full and the candidate does not beat anyone	Discard	<code>bin_full</code>

Two design choices deserve comment.

Why the source-diversity escape hatch? Without it, whichever run happened to be processed first would fill every drawer, and the dataset would silently over-represent it. The rule lets an otherwise unremarkable histogram in, purely because it comes from an unrepresented run or timestep. This is what makes the by-run train/test split in Chapter 14 meaningful.

Why replacement rather than plain rejection? A full drawer is not frozen. If a strikingly different histogram turns up late, it evicts the most redundant occupant. This makes the final contents much less dependent on processing order than a simple first-come-first-served cap would be.

Watch out. The algorithm is *greedy* and therefore order-dependent. Running with a different number of MPI ranks changes the order in which candidates reach the merge, which can change the final selection slightly. The dataset is reproducible for a fixed rank count, not across rank counts.

8.5 Bookkeeping when a replacement happens

Replacement is fiddly, because the source tally must stay correct. Inside `process_work_units_on_rank` [MPI:640] the sequence at [MPI:748] is:

1. Look up the evicted example's own source key.
2. Decrement that source's count in the drawer, and delete the entry entirely if it hits zero — otherwise “already represented” would stay true for a source that is no longer present.
3. Overwrite the metadata, the histogram, and the bin key in the three parallel lists.
4. Increment the new example's source count.

The three lists `selected_metadata`, `selected_pdfs`, and `selected_bin_keys` are index-aligned: position n in one always corresponds to position n in the others. The dictionary `bin_to_indices` maps each drawer to the list of positions it owns.

Chapter 9

Stage 8: Running on Many Processors

9.1 Why parallelism is needed and why it is awkward here

The production run examined 2.16 million windows. Each one requires reading a 262,144-cell sub-cube, computing five moments, building a 4096-cell histogram, and comparing it against up to 100 stored histograms. On a single core that takes days.

The obvious fix is to give each processor a slice of the work. But the retention algorithm is *stateful*: whether to keep example number 900,000 depends on the 899,999 decisions before it. That is exactly what does not parallelise.

The program’s answer is a two-phase design, spelled out in its opening comment [MPI:14]: run the greedy algorithm independently on each processor with a *deliberately loosened* cap, then replay all the survivors through the same algorithm one final time with the real cap.

In everyday terms. Fifty judges each screen their own pile of audition tapes and forward their top hundred. A single head judge then watches only those forwarded tapes and picks the final fifty per category. The head judge’s decisions are consistent, but they only had to watch a small fraction of the original material.

9.2 Phase 1: independent greedy selection

Splitting up the work

`enumerate_work_units` [MPI:628] builds the list of all (`run_id`, `scalar_config`) pairs — $27 \times 8 = 216$ of them for the production configuration. `partition_work_units` [MPI:636] then hands out one in every `size` to each processor:

```
[unit for index, unit in enumerate(units) if index % size == rank]
```

Here `rank` is this processor’s identity number (0, 1, 2, ...) and `size` is how many processors there are in total; both come from MPI at [MPI:1062]. This “deal the cards round-robin” scheme means consecutive units go to *different* processors, which balances the load better than handing out contiguous blocks would — runs from the same simulation often take similar amounts of time, so you want them spread around.

Watch out. The unit of work is a whole (`run`, `configuration`) pair, including all its timesteps. So the maximum useful number of processors is 216. Asking for more simply leaves the extras idle; they will still write an empty manifest so the merge knows they ran [MPI:782].

The inflated cap

Each processor runs the ordinary greedy algorithm over its slice, but with a per-drawer cap of

$$\text{per_rank_cap} = \text{max_per_moment_bin} \times \text{per_rank_cap_multiplier}$$

computed at [MPI:1100], with a floor of 1. The default multiplier is 2, so each processor keeps up to 100 per drawer instead of 50.

The reason is a subtle statistical one. If each processor kept only 50 per drawer, then for a drawer where processor 7 happens to hold all the world’s best candidates, the merge would only ever see 50 of them — and processors 0–6 would each contribute 50 mediocre ones that crowd the drawer. Over-collecting gives the final merge a richer pool to choose from. The trade-off is disk space and merge time; the option is documented as “1=tightest, 2=safe, 3=loose” [MPI:619].

What each processor writes

`write_rank_outputs` [MPI:766] saves the processor’s survivors under `<output-dir>/_rank/rank_NNNN/`, using the same writer as the final dataset (`materialize_dataset` [MPI:533]) so the format is identical. It adds three extra fields to the per-rank manifest: the processor’s rank number, the list of work units it handled, and the cap it used.

The empty-result branch [MPI:782] is important: even a processor that retained nothing writes a manifest, so that Phase 2 can tell “ran and found nothing” apart from “crashed”.

Freeing memory before the barrier

At [MPI:1162] the code explicitly deletes its Phase-1 lists before waiting at the barrier:

```
del rank_selected_metadata, rank_selected_pdfs
```

The comment above it explains why. When many processors share one physical machine, a processor that finishes early would otherwise sit idle at the barrier still holding gigabytes of histograms, starving its slower neighbours. Releasing the memory first is a courtesy that prevents out-of-memory failures late in a run.

The barrier

`comm.Barrier()` at [MPI:1164] is a rendezvous point: every processor stops there until all of them have arrived. It guarantees that all per-rank files are completely written before the merge starts reading them. Immediately after the barrier, every processor except rank 0 simply returns and exits [MPI:1166].

9.3 Phase 2: the single-processor merge

Only rank 0 continues. It globs the `rank_*` directories in sorted order [MPI:1179], so the merge order is deterministic for a given rank count.

Streaming the candidates back in

`iter_rank_candidates` [MPI:831] is a generator that yields `(metadata, histogram)` pairs from every processor’s output in turn. It is written for speed, and the docstring names the two things it deliberately avoids:

- **No `pandas.iterrows()`.** That method rebuilds a Series object for every row and is notoriously slow. Instead the metadata is converted once, in bulk, with `to_dict(orient="records")` [MPI:876], producing plain Python dictionaries.

- **No per-candidate HDF5 reads.** Each compressed shard is decompressed once into memory as a whole array [MPI:871], then indexed cheaply. Reading one histogram at a time from a gzip-compressed file would decompress the surrounding chunk every single time.

It also does some hygiene: it drops the per-rank bookkeeping columns (`sample_id`, `shard_id`, `local_index`, `hdf5_file`) which are about to be regenerated, and it drops the *stale* decision columns from Phase 1 (`moment_bin_*`, `shape_novelty`, `retain_reason`) [MPI:842], because the merge is about to recompute all of them under the global cap. Without that, the final dataset would carry misleading provenance.

Replaying the decision with the real cap

`merge_per_rank_outputs` [MPI:881] re-implements the same four-way rule, inline rather than by calling `decide_retention`, so it can reuse intermediate results. Starting at [MPI:909], for each candidate:

1. Recompute the bin key from the five moments.
2. If the drawer is empty, keep immediately, novelty recorded as 1.0.
3. Otherwise stack the drawer's histograms and compute all distances in one shot with `js_distance_to_stack` [MPI:944]; the minimum is the candidate's novelty.
4. If the drawer has room, apply the novel-shape / source-diversity / redundant test.
5. If the drawer is full, compare against the least-novel occupant and possibly replace.

The novelty cache: turning $O(N^2)$ into $O(N)$

The expensive step is “how novel is each existing occupant?”, because computing it from scratch means comparing all 50 stored histograms against each other — 1225 comparisons — and doing that for every one of 825,000 candidates would be hopeless.

The dictionary `bin_novelties` caches that array per drawer. The maintenance logic is the cleverest part of the file:

- **On admission to a non-full drawer** [MPI:966]: the new arrival can only *reduce* each existing occupant's nearest-neighbour distance, and the amount by which it could reduce it is exactly the distance we already computed. So the cached array is updated with an element-wise minimum and the new item's own novelty is appended. No recomputation at all.
- **On first encountering a full drawer** [MPI:977]: compute the array once with `pairwise_min_distances` and cache it.
- **On replacement** [MPI:999]: throw the cache away. When an occupant is evicted, its neighbours' nearest-neighbour distances can *increase*, and there is no cheap way to know by how much. The comment in the code says exactly this. Correctness is chosen over speed here.

Progress reporting

Every `--merge-progress-every` candidates (default 10,000) the loop prints how many it has seen, how many it has retained, how many drawers are populated, how many have cached novelty arrays, the current rate in candidates per second, and elapsed time [MPI:1005]. On a run that takes hours this is the only way to tell whether it is progressing or stuck.

9.4 Combining the accounting

`aggregate_rank_manifests` [MPI:1024] reads every per-rank manifest and:

- sums all the counters, prefixing each name with `phase1_` so Phase-1 and Phase-2 statistics never collide in the final manifest;
- concatenates every processor's `missing_paths` list;
- builds a per-rank summary of retained count, elapsed time, and work units handled.

In the production manifest these appear as `phase1_raw_records` (2,156,544) versus `candidates_seen` (825,249) — the first is how many windows were examined, the second how many survived Phase 1 and reached the merge.

9.5 The debugging shortcut

`--skip-phase1` [MPI:1109] lets you jump straight to the merge over an existing `_rank/` tree. Because only rank 0 does merging, you run this with plain `python`, no `mpirun` needed. This is invaluable for tuning `--max-per-moment-bin` or `--shape-threshold`: the expensive part (reading DNS files and computing two million histograms) is already done and cached on disk, so you can iterate on the selection policy in minutes rather than days. Remember to have used `--keep-rank-outputs` on the original run, otherwise the tree is deleted at the end [MPI:1216].

Chapter 10

Stage 9: What Gets Written to Disk

`materialize_dataset` [MPI:533] writes three kinds of file. The split is deliberate: big numeric arrays go in a format built for big numeric arrays, small searchable facts go in a format built for querying, and the settings go in a format a human can read.

10.1 The histograms: HDF5 shards

`write_hdf5_shards` [MPI:462] writes the histograms in chunks of at most `--hdf5-shard-size` examples (default 25,000). The production run produced seven files.

Each file contains:

Path inside file	Shape	Meaning
/bins/edges_a	(65,)	Bin edges along z_1
/bins/edges_b	(65,)	Bin edges along z_2
/bins/centers_a	(64,)	Bin centres along z_1
/bins/centers_b	(64,)	Bin centres along z_2
/bins/simplex_mask	(64,64)	1 inside the triangle, 0 outside
/bins/cell_area	(64,64)	Area of each rectangle
/data/histograms	(n , 64, 64)	The histograms themselves, 32-bit floats
/data/sample_id	(n ,)	Global identifier of each example
/data/local_index	(n ,)	Position within this file

Three storage choices are worth explaining:

- **The bin definition is repeated in every shard.** It costs a few kilobytes and makes each file self-describing — you can hand someone one shard and they can plot it correctly without any other file.
- **Chunking.** `chunks=(min(256, n), 64, 64)` means the file is physically stored in blocks of up to 256 histograms. Reading one histogram decompresses its whole block, so sequential access is fast and random access is tolerable.
- **shuffle=True before gzip.** The shuffle filter regroups the bytes of the floating-point numbers so that all the first bytes sit together, all the second bytes together, and so on. Neighbouring values in a histogram have similar exponents, so this creates long runs of identical bytes and gzip compresses them far better.

10.2 The facts about each example: Parquet

`build_metadata_frame` [MPI:513] assembles one table row per retained example and writes it as

a Parquet file — a columnar format that lets you read just the two columns you care about out of forty without touching the rest.

It inserts four locator columns at the front: `sample_id` (a simple running number), `shard_id`, `local_index`, and `hdf5_file`. Together the last two are the address of the histogram: “open this file, take row number that”. Only the *basename* of the HDF5 file is stored, not the full path, so the dataset directory can be moved or copied anywhere without breaking.

The remaining columns are everything the metadata dictionary accumulated: the identifiers, the window geometry, the ten direct-and-histogram moments, the maximum moment discrepancy, the five bin indices plus a joined string key, the shape novelty, the retention reason, and the ten shape features.

Watch out. `materialize_dataset` [MPI:533] calls `remove_existing_shards` [MPI:503] before writing [MPI:537]. If you point two different runs at the same output directory with the same `--dataset-tag`, the first one’s shards are deleted. Use distinct tags.

10.3 The settings: JSON manifest

The manifest [MPI:557] records every option that shaped the dataset — input folder, run ids, configurations, box widths, strides, bin count, z_{st} , timestep range, moment bin counts, the per-bin cap, the shape threshold — plus the complete counter tally, the number of retained examples, the list of missing paths, and the elapsed time. The MPI driver adds `mpi_size`, `per_rank_cap`, `per_rank_cap_multiplier`, and the per-rank summaries [MPI:1191].

Treat this file as the dataset’s birth certificate. If you cannot reproduce a result, the manifest is where you look first.

Chapter 11

The Idea Behind the Neural Network

11.1 Restating the goal

We now have 165,895 examples, each pairing:

- **input:** four or five moments (mean A, variance A, mean B, variance B, and optionally the covariance), and
- **output:** a 64×64 histogram, of which about 2200 cells lie inside the triangle.

We want a function that turns the input into the output.

11.2 Why not just predict the 2200 numbers directly?

That is the obvious approach, and an earlier script in this workspace (`MLPDF.py`) does exactly that. It has three serious drawbacks:

1. **Size.** The last layer alone needs $256 \times 2200 \approx 563,000$ parameters. The Dirichlet network described below uses about *one twelfth* of that in total.
2. **No guarantees.** Nothing stops the network from outputting a histogram whose mean disagrees with the mean you fed it. Since the mean is a hard physical constraint supplied by the LES solver, that is a real problem.
3. **Resolution lock-in.** A network trained to emit 64×64 values can only ever emit 64×64 values. You cannot ask it for the probability at an arbitrary point, and you cannot re-use it on a finer grid.

11.3 The alternative: predict a formula, not a picture

Instead of predicting 2200 numbers, predict the handful of parameters of a mathematical *formula* that describes the whole distribution. Then you can evaluate that formula anywhere, at any resolution, and its mean and variance can be computed exactly by algebra.

In everyday terms. Rather than sending someone a photograph of a circle as a million pixels, send them the centre and the radius. Three numbers instead of a million, and they can draw it at any size they like.

The formula chosen here is a **Dirichlet mixture**, and the network that predicts its parameters is called a **Mixture Density Network**, or MDN.

11.4 What a Dirichlet distribution is

A **Dirichlet distribution** is a probability distribution over exactly our triangle. It takes three positive numbers as parameters, written $\alpha_1, \alpha_2, \alpha_3$, and assigns to every point (z_1, z_2, z_3) in the triangle the density

$$p(z \mid \alpha) = \frac{1}{B(\alpha)} z_1^{\alpha_1-1} z_2^{\alpha_2-1} z_3^{\alpha_3-1}, \quad B(\alpha) = \frac{\Gamma(\alpha_1)\Gamma(\alpha_2)\Gamma(\alpha_3)}{\Gamma(\alpha_1 + \alpha_2 + \alpha_3)}.$$

The function Γ (“gamma”) is a standard mathematical function that extends the factorial to non-integers; you never need to compute it yourself, and the code uses the logarithm of it, `torch.lgamma`, for numerical safety.

You can read off the behaviour without any calculus:

Parameter setting	Resulting shape
All three α_i equal to 1	Perfectly flat: every point in the triangle equally likely.
All three large and equal	A tight blob at the centre. The larger they are, the tighter.
All three below 1	Mass flees to the corners — the distribution describes an <i>unmixed</i> state.
One large, two small	A blob near the corresponding corner.
Two moderate, one tiny	Mass stretched along one edge.

The sum $\alpha_0 = \alpha_1 + \alpha_2 + \alpha_3$ controls tightness (large α_0 = concentrated), and the ratios α_i/α_0 control *where* the mass sits. In fact those ratios are exactly the mean:

$$\mathbb{E}[Z_i] = \frac{\alpha_i}{\alpha_0}, \quad \text{Var}[Z_i] = \frac{\mathbb{E}[Z_i](1 - \mathbb{E}[Z_i])}{\alpha_0 + 1}, \quad \text{Cov}[Z_1, Z_2] = \frac{-\alpha_1\alpha_2}{\alpha_0^2(\alpha_0 + 1)}.$$

Note the covariance is always negative: on a triangle, more of one ingredient necessarily means less of another. This is a real physical property that the Dirichlet gets right for free.

Why the Dirichlet is the natural choice here

It lives on exactly the right domain (the triangle), it automatically enforces $z_1 + z_2 + z_3 = 1$ and non-negativity, its mean and variance are simple algebraic expressions, and it is the standard single-scalar model in combustion (the Beta distribution) generalised to two scalars.

11.5 Why a mixture of them

One Dirichlet has only three free numbers, so it can only make simple shapes: one blob, or mass pushed to the corners. Real filtered PDFs are often **bimodal** — two separate blobs, one where the fluid is still unmixed and one where it has reacted. A single Dirichlet cannot do that.

The fix is to add several together, each with its own weight:

$$p(z) = \sum_{k=1}^K \pi_k p(z \mid \alpha^{(k)}), \quad \pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$

With $K = 16$ components you have $16 \times 3 = 48$ concentration numbers plus 16 weights, 64 numbers in total — still 35 times fewer than the 2200 histogram cells, but flexible enough to represent essentially any shape that appears in the data.

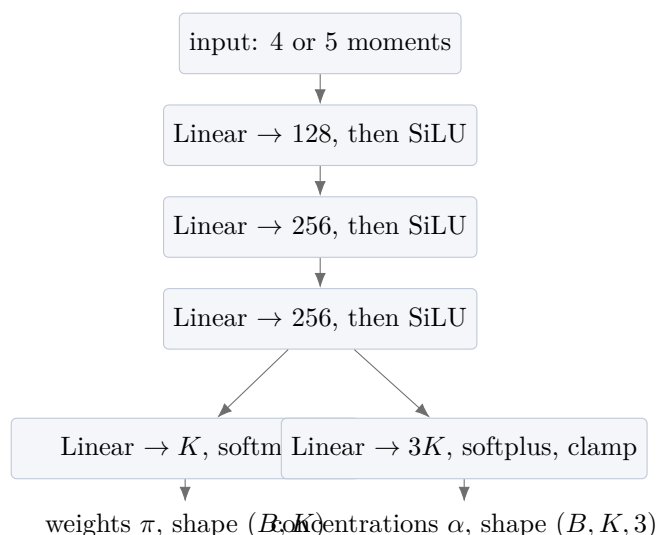
In everyday terms. A single Dirichlet is one spotlight on a stage. A mixture is sixteen spotlights, each with its own position, spread, and brightness. Together they can light up any pattern you like.

Chapter 12

The Network Itself

12.1 Architecture

The entire model is one class, `DirichletMDN` [MODEL:22], and it is deliberately small.



The shared stack of three layers is called the **trunk**, `self.trunk` [MODEL:45]. It turns the handful of input moments into 256 internal numbers that both output heads can use. The two heads then read those same 256 numbers and each produce a different part of the answer.

What a Linear layer does. It multiplies its inputs by a matrix of learned numbers and adds a learned offset. That is all — it is a weighted sum.

What SiLU does. If you stack linear layers directly, the whole stack collapses into a single linear layer and can only represent straight-line relationships. You need a **nonlinearity** between them. SiLU (also called Swish) is $f(x) = x \cdot \sigma(x)$ where σ is the S-shaped logistic curve. It passes large positive values through nearly unchanged, squashes large negative values towards zero, and — unlike the older ReLU — is smooth everywhere, which makes the training gradients better behaved.

12.2 The two output heads

The two heads have different jobs, so they enforce different constraints.

Head π : the mixture weights. `self.head_pi` [MODEL:52] produces K raw numbers, which `torch.softmax` [MODEL:63] converts into K non-negative numbers that sum to 1:

$$\pi_k = \frac{e^{u_k}}{\sum_{j=1}^K e^{u_j}}.$$

Exponentiating makes everything positive; dividing by the total makes them sum to one. So the weights are valid by construction — the network cannot produce an invalid answer even before training.

Head α : the concentrations. `self.head_alpha` [MODEL:53] produces $3K$ raw numbers. These must be strictly positive, so they pass through **softplus**, `softplus(x) = log(1 + ex)`, a smooth function that is always positive and approaches x for large x . Then:

- A floor `alpha_min` (default 10^{-3}) is *added*. Softplus can return values arbitrarily close to zero, and α near zero makes the log-density explode. The floor keeps the arithmetic finite.
- A ceiling `alpha_clip` (default 10^3) is applied with `clamp`. Very large α means an extremely narrow spike, which produces enormous gradients and destabilises training.
- The flat vector of $3K$ numbers is reshaped to $(B, K, 3)$ so that component k owns rows `alpha[:, k, :]`.

The constructor also validates its arguments up front [MODEL:32]: `n_in` must be 4 or 5, K must be at least 1, and the ceiling must exceed the floor.

12.3 Size

With $K = 16$, `hidden = 256`, and 5 input moments:

$$\underbrace{5 \cdot 128 + 128}_{\text{layer 1}} + \underbrace{128 \cdot 256 + 256}_{\text{layer 2}} + \underbrace{256 \cdot 256 + 256}_{\text{layer 3}} + \underbrace{256 \cdot 16 + 16}_{\text{head } \pi} + \underbrace{256 \cdot 48 + 48}_{\text{head } \alpha} \approx 116,000 \text{ parameters.}$$

That is small enough to run inside an LES solver's inner loop, which was the point.

Chapter 13

How the Network Learns

Training means adjusting those 116,000 numbers until the predictions match the data. What counts as “match” is defined by the **loss** — a single number, computed from a batch of examples, that is small when the predictions are good. Training is then just: compute the loss, work out which direction each parameter should move to reduce it, take a small step, repeat.

The loss here is a sum of three parts `combined_loss` [LOSS:261]:

$$L_{\text{total}} = \underbrace{L_{\text{nll}}}_{\text{match the picture}} + \lambda_{\text{mom}} \underbrace{L_{\text{mom}}}_{\text{match the numbers}} - \lambda_{\text{ent}} \underbrace{H}_{\text{stay diverse}}.$$

13.1 Part 1: matching the histogram

`HistogramNLL` [LOSS:80] answers: “if the true histogram were a bag of dots, how surprised would my predicted formula be to see them?” Lower surprise, better fit.

Precomputation in the constructor

Everything that does not change from batch to batch is computed once when the module is created [LOSS:85]:

1. Flatten the 64×64 grid and keep only the in-triangle cells, giving $M \approx 2200$ points. Store their coordinates as `z_simplex` of shape $(M, 3)$, with the third coordinate reconstructed as $z_3 = 1 - z_1 - z_2$, clamped at zero.
2. Store the cell areas of those M cells, and their logarithms as `log_cell_area`.
3. Store log of the normalised coordinates as `log_z_norm`, pre-shaped to $(1, M, 1, 3)$ so it can be broadcast against α without any reshaping at run time.
4. Store the flat indices of the in-triangle cells, so a full $(B, 64, 64)$ histogram can be reduced to (B, M) with one gather.

All of these are registered as **buffers**, which means they automatically move to the GPU when the model does but are not treated as trainable parameters.

The forward computation

For a batch of B examples [LOSS:132]:

1. **Log-density of every component at every cell.** Written out, the Dirichlet log-density is

$$\log p(z_m | \alpha^{(k)}) = \sum_{i=1}^3 (\alpha_i^{(k)} - 1) \log z_{m,i} - \sum_{i=1}^3 \log \Gamma(\alpha_i^{(k)}) + \log \Gamma\left(\sum_{i=1}^3 \alpha_i^{(k)}\right).$$

The first term is a dot product against the precomputed `log_z_norm`; the other two are `lgamma` calls. Result shape: (B, M, K) .

2. **Combine the components.** Add $\log \pi_k$ and use `logsumexp` over k :

$$\log p(z_m) = \log \sum_k \pi_k p(z_m | \alpha^{(k)}).$$

`logsumexp` computes $\log \sum e^{x_i}$ without ever forming e^{x_i} , by factoring out the largest term. Doing it naively would overflow to infinity or underflow to zero within a few epochs.

3. **Density to probability.** Multiply by the cell area — in log space, add `log_cell_area`.
4. **Renormalise over the triangle.** Subtract another `logsumexp`, this time over the cells. This is the crucial step: the true histogram sums to 1 over the triangle, so the prediction must too, even though a Dirichlet formally spreads a little mass outside the discretised region.
5. **The surprise.** Multiply the true histogram by the predicted log-probabilities, sum, and negate:

$$L_{\text{nll}} = -\frac{1}{B} \sum_{b=1}^B \sum_{m=1}^M p_{b,m} \log q_{b,m}.$$

In everyday terms. Imagine the true histogram is a list of guesses people made, and the prediction is a bookie's odds. This loss is the total "I can't believe that happened" score. Good odds give a low score.

An optimisation worth knowing about

The forward pass accepts either a full $(B, 64, 64)$ histogram or an already-flattened (B, M) one [LOSS:148], deciding by checking the tensor's shape. If you pass `--flat-histograms` on the training command line, the dataset does the flattening once at load time [DATA:182] and the loss skips a reshape-and-gather on every single batch.

13.2 Part 2: matching the moments exactly

The histogram loss cares about the whole picture. But we also want the prediction's mean and variance to agree with the numbers we fed in — because in deployment those numbers are hard constraints from the physics solver.

The wonderful thing about a Dirichlet mixture is that its moments can be computed by algebra, with no sampling and no integration. `mixture_moments` [LOSS:165] does it in nine lines.

Per component k , with $\alpha_0^{(k)} = \sum_i \alpha_i^{(k)}$:

$$\mu_i^{(k)} = \frac{\alpha_i^{(k)}}{\alpha_0^{(k)}}, \quad v_i^{(k)} = \frac{\mu_i^{(k)}(1 - \mu_i^{(k)})}{\alpha_0^{(k)} + 1}, \quad c_{12}^{(k)} = \frac{-\alpha_1^{(k)} \alpha_2^{(k)}}{(\alpha_0^{(k)})^2 (\alpha_0^{(k)} + 1)}.$$

Combining them across the mixture needs care. The mean is a simple weighted average:

$$\mathbb{E}[Z_i] = \sum_k \pi_k \mu_i^{(k)}.$$

But the *variance* is not. You cannot just average the component variances, because the components sit in different places and that separation is itself a source of spread. The correct route is via the second moment:

$$\mathbb{E}[Z_i^2] = \sum_k \pi_k \left(v_i^{(k)} + (\mu_i^{(k)})^2 \right), \quad \text{Var}[Z_i] = \mathbb{E}[Z_i^2] - (\mathbb{E}[Z_i])^2.$$

And likewise for the covariance:

$$\mathbb{E}[Z_1 Z_2] = \sum_k \pi_k \left(c_{12}^{(k)} + \mu_1^{(k)} \mu_2^{(k)} \right), \quad \text{Cov}[Z_1, Z_2] = \mathbb{E}[Z_1 Z_2] - \mathbb{E}[Z_1] \mathbb{E}[Z_2].$$

In everyday terms. Two rifle groupings, each tight, but centred a metre apart. The combined spread is far larger than either group's own spread, because you must also account for the gap between them. That gap is the $-(\mathbb{E}[Z])^2$ correction.

`stack_predicted_moments` [LOSS:190] arranges these into the same order as the input vector — $[\mathbb{E}_1, V_1, \mathbb{E}_2, V_2]$ for the four-moment setting, with `Cov12` appended for five — and `moment_loss` [LOSS:206] takes the mean squared difference against the true moments.

This term is weighted by `--lambda-mom` (default 0.5).

Watch out. Note carefully that the moment loss compares against the *unscaled* moments, while the network sees *scaled* moments as input. Look at the training inner loop [TRAIN:327]: the scaler is applied to make `m_scaled` for the model, but the raw `moments` tensor is what is passed to the loss. Mixing these up would make the moment term meaningless.

13.3 Part 3: keeping the components alive

Mixture models have a well-known failure mode: the optimiser discovers that it can drive most of the weights π_k to nearly zero and do all the work with two or three components. The other thirteen become dead weight, and the model is effectively much less expressive than you paid for.

The cure is to reward spread-out weights. `entropy_reg` [LOSS:222] computes

$$H = \frac{1}{B} \sum_{b=1}^B \left(- \sum_{k=1}^K \pi_{b,k} \log \pi_{b,k} \right),$$

which is largest when all weights are equal and zero when one weight takes everything. Because we want H *large* and the optimiser always makes the total *small*, the term enters the sum with a minus sign [LOSS:269] — that is, we minimise $-\lambda_{\text{ent}} H$.

The default weight `--lambda-ent` is 10^{-3} : a gentle nudge, not a hard rule. Push it too high and every component is forced to be equally important whether or not that helps.

13.4 Numerical hygiene throughout

Three habits recur in the loss code and are worth internalising:

- **Everything in logarithms.** Probabilities multiply; logarithms add. Adding is numerically stable, multiplying tiny numbers is not.
- **logsumexp instead of log(sum(exp(...))).** See `mixture_logpdf` [LOSS:50] and the inline version at [LOSS:144].
- **Small constants everywhere.** `LOG_EPS = 10-12` [LOSS:30] guards $\log \pi$; `eps = 10-7` guards $\log z$ inside `dirichlet_logpdf` [LOSS:37]. Without them the very first batch containing an empty histogram cell yields NaN and the run is dead.

Chapter 14

Feeding the Network: Dataset and Splits

14.1 The dataset adapter

`HybridPDFDataset` [DATA:76] is the bridge between the files on disk and the training loop. Its guiding principle, stated in the module docstring, is **schema tolerance**: it accepts metadata written by either the old or the new version of the writer.

`load_metadata` [DATA:48] handles this by looking for `run_id` or `run_number`, and `box_width` or `filter_width`, renaming whichever it finds to the canonical name, then verifying that all twelve required columns are present and raising a clear error listing exactly which are missing if not.

A safety check you should not skip

On construction, `_check_grid_matches_first_shard` [DATA:115] rebuilds the bin grid from scratch using `bin_grid` [GRID:75] and compares it against the `/bins` group actually stored in the first HDF5 shard, demanding agreement to 10^{-9} (`validate_against_hdf5` [GRID:107]).

This matters because the training code re-derives the grid from three numbers (`--num-bins`, `--zst`, `--uniform-bins`) rather than reading it from the file. If you train with the wrong z_{st} , every histogram would be silently misinterpreted and the results would be quietly wrong. The check turns a silent disaster into a loud crash.

Fetching one example

`__getitem__` [DATA:162] does the following:

1. Look up the row: which shard file, which row inside it.
2. Open the shard lazily. Handles are cached in a dictionary [DATA:136], so the file is opened at most once even across hundreds of thousands of reads.
3. Read the histogram.
4. **Verify it sums to 1**, but only for the first read from each shard [DATA:171]. A full check on every read would be pure overhead; a check once per file catches a corrupted or wrongly-normalised shard immediately.
5. Optionally flatten to in-triangle cells; optionally store in a RAM cache.
6. Assemble the 4- or 5-element moment vector and the metadata dictionary.

Two speed options

`--cache-histograms` keeps every histogram in memory after its first read. The help text quotes about 2.6 GB for the full dataset. Since training runs hundreds of epochs, this turns hundreds of passes over compressed disk data into one.

`--flat-histograms` stores only the in-triangle cells, roughly halving both memory and per-batch work. Used together, the two options remove file input/output from the training loop almost entirely.

Batching

`collate_records` [DATA:224] stacks a list of examples into batched tensors and converts the meta-data into parallel tensors, keeping `scalar_config` as a plain list of strings since it is text.

14.2 Making an honest train/test split

This is the part that decides whether your reported accuracy means anything.

The leakage trap

Suppose you shuffled all 165,895 examples and took 15% at random for testing. Two examples from overlapping windows of the same snapshot are nearly identical — one would land in training and its near-twin in testing. Your test score would be excellent and completely meaningless, because you would only have proved that the network can recognise things it has already seen.

The fix: split by simulation run

`make_split` [SPLIT:80] splits by `run_id`, not by row. All examples from run 7 go entirely into training, or entirely into validation, or entirely into testing. Since different runs start from different random initial conditions, a test run is genuinely unseen physics.

The procedure, per scalar configuration:

- If the configuration has at least three distinct runs, `_split_runs` [SPLIT:35] shuffles the run list with a seeded generator and slices it 70/15/15 by default. Seeding means the same seed always produces the same split, so results are reproducible.
- If it has fewer than three runs, a run-based split is impossible. The fallback is chosen by `--split-fallback`:
 - `timestep` (default) uses `_split_timesteps` [SPLIT:57] to cut the timesteps into three *contiguous, non-shuffled* blocks — early ones for training, later ones for validation and testing. Contiguity matters: turbulence evolves smoothly, so interleaving timesteps would leak information forwards and backwards in time.
 - `train_only` puts everything into training and leaves that configuration out of validation and testing entirely.
- If `--holdout-config` names a configuration, that entire configuration becomes the test set and everything else is split 80/20 between training and validation. This is the harshest and most informative test: can the model generalise to an initial condition it has never encountered?

Whatever route is taken, the `description` dictionary records the mode used per configuration, the number of runs, the number of rows, and the exact run or timestep lists. That record is saved alongside the model, so months later you can prove which runs the model never saw.

You can inspect a split before committing to it with `python -m dirichlet_mdn.preview_split`, which writes a Markdown table and a CSV summary; see `splits/by_run_id_seed0/` in this workspace for an example.

Chapter 15

The Training Loop

15.1 Setup

`train` [TRAIN:213] performs, in order:

1. Seed every random number generator so the run is reproducible.
2. Create a run directory named with a timestamp and the `--tag` you supplied, so runs never overwrite each other.
3. Load the metadata and either build a split or load a pre-computed one from `--splits-json`.
4. **Fit the input scaler** [TRAIN:265] on the *training rows only*.
5. Build the model, the loss module, and the data loaders.
6. Create the optimiser and the learning-rate schedule.

Why scale the inputs, and why a robust scaler

The five moments live on wildly different scales: means are around 0.1–0.9, variances around 0.001–0.05, covariance can be negative. A neural network trained on such mismatched inputs converges slowly, because the same step size is far too big for one input and far too small for another.

The fix is to standardise each input. This code uses scikit-learn’s `RobustScaler`, which subtracts the *median* and divides by the *interquartile range* (the width of the middle 50% of the data), rather than the more common mean-and-standard-deviation. The reason is outliers: a handful of extreme variance values would badly distort a mean-based scaler, while the median barely notices them.

`TorchScaler` [TRAIN:105] wraps the fitted scaler so it can be applied directly to a GPU tensor, avoiding a round trip to the CPU on every batch. It also guards against division by zero by replacing any scale below 10^{-6} with 1.0 [TRAIN:114].

Watch out. The scaler is fitted on training rows only, then applied unchanged to validation and test rows. Fitting on the full dataset would leak information about the test distribution into training. The fitted scaler is saved to `scaler.pkl` and must accompany the model at deployment.

15.2 The optimiser and the schedule

[TRAIN:292] creates `AdamW`. In plain terms, Adam keeps a running memory of both the recent direction of improvement and the recent size of the changes, and uses them to give each parameter

its own effective step size. The “W” variant additionally applies **weight decay** — a gentle pull of every parameter towards zero — in the mathematically correct way. Weight decay discourages the network from relying on any single huge parameter, which usually improves generalisation. Defaults: learning rate 3×10^{-4} , weight decay 10^{-5} .

[TRAIN:293] adds **CosineAnnealingLR**. The learning rate starts at its full value and follows a cosine curve down to nearly zero by the last epoch. Big steps early explore the landscape; tiny steps late settle precisely into a minimum.

15.3 One epoch

An **epoch** is one complete pass over the training data. For each batch, starting at [TRAIN:314]:

1. Move the moments and the histogram to the compute device.
2. Apply the scaler to produce `m_scaled`.
3. Forward pass: `pi, alpha = model(m_scaled)` [TRAIN:327].
4. Compute the three-part loss against the *raw* moments and the true histogram.
5. Zero the accumulated gradients, then back-propagate `loss.total`.
6. **Clip the gradients** to a maximum overall length of `--grad-clip` (default 5.0) [TRAIN:334]. Occasionally a pathological batch produces an enormous gradient that would throw the parameters into nonsense; clipping caps the step size while preserving its direction.
7. Take one optimiser step.
8. Every `--log-every-n-batches` batches (default 25), print progress.

After the batches, step the learning-rate schedule and run `evaluate_loss` [TRAIN:133] over the validation set with gradients disabled.

15.4 Choosing when to stop

- **Best-model tracking.** If the validation negative-log-likelihood improves, the checkpoint `model_best.pt` is rewritten [TRAIN:409]. Note the criterion is the NLL term alone, not the combined total — fitting the shape of the distribution is the primary objective.
- **Early stopping.** A counter tracks epochs without improvement. When it reaches `--patience` (default 30), training halts [TRAIN:421]. Set patience to 0 to disable.
- **Last-model checkpoint.** `model_last.pt` is always written too, so you can inspect the end state even if it overfitted.

15.5 What a run directory contains

File	Contents
<code>manifest.json</code>	Every configuration value, dataset statistics, split totals, best score
<code>splits.json</code>	The exact row indices of each split, plus the provenance description
<code>scaler.pkl</code>	The fitted input scaler — required to use the model
<code>model_best.pt</code>	Weights at the best validation NLL
<code>model_last.pt</code>	Weights at the final epoch
<code>tb/</code>	TensorBoard event files for live curves
<code>train.log</code>	Per-epoch loss table, also echoed to the terminal

The existing runs under `dirichlet_mdn_runs/` follow this layout exactly; see for example `20260618_145502_mpi_M5_K16_500ep_hidden256/`.

15.6 Devices

`resolve_device`^[TRAIN:90] maps `--device auto` to an NVIDIA GPU if present, otherwise to Apple Silicon’s Metal backend, otherwise to the CPU. Because the model is tiny, CPU training is perfectly viable for small experiments.

Chapter 16

Judging the Result

16.1 Turning parameters back into a picture

`predict_histograms` [METRIC:28] does the reverse of the loss: given π and α , it evaluates the mixture at every in-triangle cell, multiplies by cell area, renormalises, and returns a $(B, 64, 64)$ array directly comparable with the stored histogram. Cells outside the triangle are left at zero.

16.2 The four numbers that matter

L_1 error

`joint_l1` [METRIC:72] sums the absolute cell-by-cell differences, $\sum_{ij} |p_{ij} - q_{ij}|$, over the triangle. Because both tables sum to 1, this ranges from 0 (identical) to 2 (no overlap at all). Half of it is the fraction of probability mass sitting in the wrong place — an L_1 of 0.2 means 10% of the mass is misplaced. It is the easiest metric to explain to a non-specialist.

Jensen–Shannon divergence

`joint_jsd` [METRIC:79] is the same family of measure used in the sampler, computed here in *bits* (base-2 logarithms) to match the convention in the legacy `MLPDF.py`. It ranges from 0 to 1 bit. Unlike L_1 it is sensitive to relative errors, so it notices when a region that should have a small amount of probability has none at all.

Marginal divergences

`marginal_jsd` [METRIC:92] collapses the two-dimensional histogram onto one axis (summing over the other) and compares those one-dimensional profiles. This separates “is scalar A right on its own?” from “is the relationship between A and B right?”. A model can score well on both marginals and still get the joint structure wrong.

Moment recovery error

`moment_recovery_error` [METRIC:125] compares the closed-form moments of the prediction against the input moments, absolutely and relatively. This is the physics constraint: if it is not near zero, the prediction is not usable in a solver regardless of how good the picture looks.

`mixture_active_count` [METRIC:120] additionally counts how many components carry weight above 1% — the direct diagnostic for the component-collapse problem the entropy term was designed to prevent. If you train with $K = 16$ and this consistently reports 3, you are paying for capacity you are not using.

16.3 Slicing the results

Aggregate numbers hide problems. `per_config_breakdown` [METRIC:149] and `per_timestep_breakdown` [METRIC:168] report mean, standard deviation, and count for every metric, grouped by scalar configuration and by configuration-and-timestep. This is how you discover that the model is excellent everywhere except one configuration, or that accuracy degrades steadily as the flow mixes.

`evaluate_run` [EVAL:206] runs the whole pipeline and writes `metrics_per_record.csv`, `metrics_per_config.csv`, `metrics_per_config.md`, `metrics_per_timestep.csv`, and `eval_summary.json` into an `eval/` subdirectory, plus a folder of plots.

16.4 The plots

- `plot_pdf_comparison` [PLOT:46] puts the true and predicted histograms side by side as contour maps with a shared colour scale, and overlays the triangle boundary via `_add_simplex_overlay` [PLOT:30]. This is the plot to look at first — your eye will catch structural errors that no single number reports.
- `plot_metric_vs_timestep` [PLOT:84] draws error against time, one line per configuration, revealing whether the model degrades as mixing proceeds.
- `plot_alpha_diagnostics` [PLOT:108] shows the distribution of active component counts and a log-scale scatter of overall concentration against variance.
- `plot_moment_recovery` [PLOT:145] scatters predicted against target for each moment with a $y = x$ reference line. Points off the diagonal are constraint violations.
- `write_pdf_album` [PLOT:171] bundles a selection into a single multi-page PDF for review.

Chapter 17

Built-In Self-Checks

`dirichlet_mdn/verify.py` contains five tests designed to catch the specific mistakes that are easy to make and hard to notice. Run them with `pixi run verify` before trusting any result.

1. `check_bins` [VERIFY:43] — Rebuild the bin grid from the three parameters and compare against a stored shard, to 10^{-9} . Catches a wrong z_{st} or bin count, which would silently misinterpret every histogram.
2. `check_loss_gradient` [VERIFY:54] — Push eight real examples forward and backward and confirm the loss and every gradient are finite. Catches numerical blow-ups before you waste hours on a training run that will produce NaN.
3. `check_moment_loss_zero_on_truth` [VERIFY:82] — Construct a case where the predicted moments should exactly equal the targets, and confirm the moment loss is about 10^{-10} . This is a pure algebra check on `mixture_moments` [LOSS:165]; a sign error or a misplaced π_k would show up here and essentially nowhere else. It sweeps $K \in \{1, 4, 8\}$ and both input sizes.
4. `check_permutation_invariance` [VERIFY:98] — Shuffle the order of the K components and confirm the loss is unchanged to within 10^{-6} . The components are unordered by definition; if relabelling them changed the answer, there would be an indexing bug.
5. `check_k1_fits_analytical_dirichlet` [VERIFY:166] — Generate a histogram from a known single Dirichlet, fit a $K = 1$ model to it (initialised by `_method_of_moments_alpha` [VERIFY:146]), and confirm the recovered parameters are within 2% of the truth. An end-to-end proof that the machinery can recover a known answer.

Chapter 18

Running Everything

18.1 Setting up the environment

The workspace ships a `pixi.toml` at its root. `pixi` is an environment manager: it reads that file, downloads exactly the listed packages, and gives you an isolated environment. Nothing is installed system-wide and nothing conflicts with your other projects.

Command	Effect
<code>pixi install</code>	Create the environment (first time only)
<code>pixi shell</code>	Drop into a shell with everything on the path
<code>pixi task list</code>	List every available task
<code>pixi run <task></code>	Run one task

The environment includes NumPy, pandas, PyArrow, h5py, SciPy, scikit-learn, matplotlib, PyTorch, TensorBoard, `mpi4py` with OpenMPI, and `tectonic` for building this document.

18.2 The task list

Task	What it does
<code>sample-mpi <n></code>	Full MPI sampling with n processors. Both phases. Example: <code>pixi run sample-mpi 52</code>
<code>sample-mpi-smoke</code>	Two processors, two runs, one timestep — a quick plumbing test
<code>merge-only</code>	Re-run Phase 2 over an existing <code>_rank/</code> tree. No MPI needed
<code>sample-serial</code>	The single-process reference sampler
<code>verify</code>	The five self-checks
<code>diversity</code>	Dataset coverage audit with plots
<code>preview-split <seed></code>	Build and report a split without training
<code>train <K> <M> <epochs> <tag></code>	Train the network. Example: <code>pixi run train 16 5 500 mdn_K16</code>
<code>train-smoke</code>	Two epochs, to confirm the loop runs
<code>evaluate <dir></code>	Full evaluation of a finished run
<code>tensorboard</code>	Live training curves in a browser
<code>doc</code>	Rebuild this document with <code>tectonic</code>

The default paths are set in the `[activation.env]` block of `pixi.toml` (`DNS_ROOT`, `PARQUET`, `HDF5_DIR`, `RUNS_DIR`). Edit them there rather than retyping long paths.

18.3 A recommended first session

1. `pixi install`
2. `pixi run verify` — confirms the existing dataset and the code agree
3. `pixi run diversity` — look at what is actually in the dataset
4. `pixi run preview-split 0` — see which runs go where
5. `pixi run train-smoke` — two epochs, confirm the loop works
6. `pixi run train 16 5 500 my_first_run` — the real thing
7. `pixi run tensorboard` — watch it learn
8. `pixi run evaluate dirichlet_mdn_runs/<timestamp>_my_first_run`

18.4 Practical notes on the MPI sampler

- There are only 216 work units, so more than 216 processors buys nothing.
- Each processor needs roughly 1–2 GB for the padded cubes plus its growing selection. Do not oversubscribe a node’s memory.
- Always pass `--keep-rank-outputs` on a long run. Re-running the merge afterwards with `--skip-phase1` is minutes; re-running Phase 1 is days.
- Phase 2 is single-threaded by construction. On the production dataset it took about three hours. Watch the progress lines to confirm it is moving.
- Read the manifest’s `missing_paths` before using the dataset.

Glossary

Barrier	A synchronisation point where every processor waits until all have arrived.
Batch	A group of examples processed together in one training step.
Bimodal	Having two separate peaks.
Bin	One small rectangle of the histogram grid.
Covariance	A measure of whether two quantities vary together.
Dirichlet distribution	A probability distribution over the triangle, controlled by three positive numbers.
DNS	Direct Numerical Simulation: the fine, expensive reference simulation.
Early stopping	Halting training when validation performance stops improving.
Entropy	A measure of how spread out a set of probabilities is.
Epoch	One complete pass over the training data.
Filter box	A cube of DNS cells treated as one coarse cell.
Gradient	The direction and rate at which the loss changes as a parameter changes.
HDF5	A file format for large numeric arrays.
Histogram	A table of counts (here, fractions) per bin.
Jensen–Shannon distance	A symmetric, bounded measure of how different two distributions are.
LES	Large Eddy Simulation: the coarse, affordable simulation.
Linear layer	A learned weighted sum plus an offset.
Logsumexp	A numerically safe way to compute $\log \sum e^{x_i}$.
Loss	The single number training tries to minimise.
MDN	Mixture Density Network: a network that outputs the parameters of a mixture distribution.
Mean	The average.
Mixture	A weighted sum of several distributions.
Moments	The summary numbers: means, variances, covariance.

MPI	Message Passing Interface: the standard for multi-processor programs.
NLL	Negative log-likelihood: the “surprise” score.
Parquet	A columnar table file format built for fast queries.
Periodic	Wrapping around at the boundaries, like an infinite tiled pattern.
PDF	Probability density function: the shape of a distribution.
Rank	A processor’s identity number in an MPI job, from 0 upwards.
Shard	One file of a dataset split across several files.
Simplex	The triangle of allowed values.
Softmax	A function turning arbitrary numbers into positive numbers summing to 1.
Softplus	A smooth function that is always positive.
Stride	How far the sliding window moves between samples.
Variance	A measure of spread.
Weight decay	A gentle pull of parameters towards zero, to improve generalisation.

Appendix: Index of Source Locations

EnsignPDFHybridDatasetMPI.py — the MPI sampler

Line	Name	Role
41	SCALAR_CONFIGS	The eight configuration labels
42	MOMENT_NAMES	The five moment column names
43	SHAPE_NAMES	The ten shape feature names
57	parse_int_spec	Parses 0:27 or 0,3,5 into a list
80	parse_str_list	Parses a comma-separated string list
87	fill_periodic	Wraps the cube with periodic ghost cells
104	read_ensight	Reads and un-decomposes the two binary files
149	get_bin_centers	Non-uniform bin centre placement
177	get_histogram_layout	Edges, centres, areas, triangle mask
215	compute_direct_moments	Five moments from raw values
224	compute_histogram_pdf	Counts into bins, normalises to 1
237	compute_histogram_moments	Five moments from the histogram
246	build_shape_masks	Seven regional stencils
264	compute_shape_features	Ten shape descriptors
286	jensen_shannon_distance	Distance between two histograms
297	js_distance_to_stack	One against many, vectorised
323	pairwise_min_distances	Nearest-neighbour distance within a set
349	compute_moment_bin	Five moments to a five-integer bin key
369	min_distance_to_existing	Scalar loop version of the above
376	get_existing_novelties	Per-item novelty, scalar version
388	decide_retention	The four-way keep/replace/discard rule
414	iter_filter_records	The sliding-window generator
462	write_hdf5_shards	Writes histograms and bin definition
503	remove_existing_shards	Cleans stale output files
513	build_metadata_frame	Builds the Parquet table
533	materialize_dataset	Writes HDF5 + Parquet + manifest
591	build_arg_parser	All command-line options
628	enumerate_work_units	All (run, configuration) pairs
636	partition_work_units	Round-robin deal to processors
640	process_work_units_on_rank	Phase 1: per-processor greedy selection
766	write_rank_outputs	Phase 1 output, including empty manifests
831	iter_rank_candidates	Fast streaming read-back of candidates
881	merge_per_rank_outputs	Phase 2: global merge with novelty cache
1024	aggregate_rank_manifests	Sums counters, collects missing paths

Line	Name	Role
1047	main	MPI setup, phase orchestration, final write

dirichlet_mdn/ — the neural network package

File & name	Line	Role
bin_grid.BinGrid	22	Container for the grid definition
bin_grid._bin_centers_1d	38	Mirrors the sampler's bin placement
bin_grid._edges_from_centers	65	Midpoint edges with extrapolated ends
bin_grid.bin_grid	75	Builds the full 2-D grid
bin_grid.validate_against_hdf5	107	Confirms agreement with a shard
data.load_metadata	48	Schema-tolerant Parquet loader
data.HybridPDFDataset	76	One example = one filter box
data.collate_records	224	Batches examples into tensors
splits.SplitResult	28	Split indices plus provenance
splits._split_runs	35	Seeded shuffle-and-slice by run
splits._split_timesteps	57	Contiguous fallback split
splits.make_split	80	The leak-free split policy
splits.save_split	211	Writes splits.json
splits.load_split	223	Reads splits.json
model.DirichletMDN	22	Trunk plus two output heads
losses.dirichlet_logpdf	37	Single Dirichlet log-density
losses.mixture_logpdf	50	Mixture log-density via logsumexp
losses.HistogramNLL	80	The histogram matching loss
losses.mixture_moments	165	Closed-form mixture moments
losses.stack_predicted_moments	190	Arranges moments to match the input
losses.moment_loss	206	Mean squared moment error
losses.entropy_reg	222	Mixture-weight entropy
losses.LossWeights	233	Default λ values
losses.combined_loss	261	The three-part total
train.TrainConfig	53	Every training setting
train.resolve_device	90	auto $\rightarrow$ CUDA / MPS / CPU
train.TorchScaler	105	GPU-friendly input scaling
train.evaluate_loss	133	Validation pass, no gradients
train.write_manifest	184	Records the run's configuration
train.setup_logger	199	Log to file and terminal
train.train	213	The full training procedure
train.build_argparser	451	All command-line options
metrics.predict_histograms	28	Renders the mixture onto the grid
metrics.joint_l1	72	Absolute error over the triangle
metrics.joint_jsd	79	Divergence in bits
metrics.marginal_jsd	92	Divergence of one-dimensional profiles
metrics.mixture_active_count	120	Components above 1% weight
metrics.moment_recovery_error	125	Physics constraint violation
metrics.per_config_breakdown	149	Statistics by configuration

File & name	Line	Role
metrics.per_timestep_breakdown	168	Statistics by configuration and time
evaluate._load_context	65	Restores model, scaler, grid
evaluate._gather_predictions	109	Runs the test set through the model
evaluate._build_metrics_df	178	Assembles the per-record metric table
evaluate.evaluate_run	206	Full evaluation and report writing
plotting._add_simplex_overlay	30	Draws the triangle boundary
plotting.plot_pdf_comparison	46	True versus predicted, side by side
plotting.plot_metric_vs_timestep	84	Error over time per configuration
plotting.plot_alpha_diagnostics	108	Component usage diagnostics
plotting.plot_moment_recovery	145	Predicted versus target moments
plotting.write_pdf_album	171	Multi-page PDF of figures
verify.check_bins	43	Grid agreement check
verify.check_loss_gradient	54	Finite loss and gradients
verify.check_moment_loss_zero_on_truth	82	Moment algebra check
verify.check_permutation_invariance	98	Component-order independence
verify.check_k1_fits_analytical_dirichlet	166	End-to-end recovery test